

Microcontroller Based Applied Digital Control

Microcontroller Based Applied Digital Control

Dogan Ibrahim

*Department of Computer Engineering
Near East University, Cyprus*



John Wiley & Sons, Ltd

Copyright © 2006 John Wiley & Sons Ltd, The Atrium, Southern Gate, Chichester,
West Sussex PO19 8SQ, England
Telephone (+44) 1243 779777

Email (for orders and customer service enquiries): cs-books@wiley.co.uk
Visit our Home Page on www.wiley.com

All Rights Reserved. No part of this publication may be reproduced, stored in a retrieval system or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, scanning or otherwise, except under the terms of the Copyright, Designs and Patents Act 1988 or under the terms of a licence issued by the Copyright Licensing Agency Ltd, 90 Tottenham Court Road, London W1T 4LP, UK, without the permission in writing of the Publisher. Requests to the Publisher should be addressed to the Permissions Department, John Wiley & Sons Ltd, The Atrium, Southern Gate, Chichester, West Sussex PO19 8SQ, England, or emailed to permreq@wiley.co.uk, or faxed to (+44) 1243 770620.

Designations used by companies to distinguish their products are often claimed as trademarks. All brand names and product names used in this book are trade names, service marks, trademarks or registered trademarks of their respective owners. The Publisher is not associated with any product or vendor mentioned in this book.

This publication is designed to provide accurate and authoritative information in regard to the subject matter covered. It is sold on the understanding that the Publisher is not engaged in rendering professional services. If professional advice or other expert assistance is required, the services of a competent professional should be sought.

Other Wiley Editorial Offices

John Wiley & Sons Inc., 111 River Street, Hoboken, NJ 07030, USA

Jossey-Bass, 989 Market Street, San Francisco, CA 94103-1741, USA

Wiley-VCH Verlag GmbH, Boschstr. 12, D-69469 Weinheim, Germany

John Wiley & Sons Australia Ltd, 42 McDougall Street, Milton, Queensland 4064, Australia

John Wiley & Sons (Asia) Pte Ltd, 2 Clementi Loop #02-01, Jin Xing Distripark, Singapore 129809

John Wiley & Sons Canada Ltd, 22 Worcester Road, Etobicoke, Ontario, Canada M9W 1L1

Wiley also publishes its books in a variety of electronic formats. Some content that appears in print may not be available in electronic books.

Library of Congress Cataloging-in-Publication Data

Ibrahim, Dogan.

Microcontroller based applied digital control / Dogan Ibrahim.

p. cm.

ISBN 0-470-86335-8

1. Process control—Data processing. 2. Digital control systems—Design and construction. 3. Microprocessors.

I. Title.

TS156.8.I126 2006

629.8'9—dc22

2005030149

British Library Cataloguing in Publication Data

A catalogue record for this book is available from the British Library

ISBN-13 978-0-470-86335-0 (HB)

ISBN-10 0-470-86335-8 (HB)

Typeset in 10/12pt Times by TechBooks, New Delhi, India

Printed and bound in Great Britain by Antony Rowe Ltd, Chippenham, Wiltshire

This book is printed on acid-free paper responsibly manufactured from sustainable forestry in which at least two trees are planted for each one used for paper production.

Contents

Preface	xi
1 Introduction	1
1.1 The Idea of System Control	1
1.2 Computer in the Loop	2
1.3 Centralized and Distributed Control Systems	5
1.4 Scada Systems	6
1.5 Hardware Requirements for Computer Control	7
1.5.1 General Purpose Computers	7
1.5.2 Microcontrollers	8
1.6 Software Requirements for Computer Control	9
1.6.1 Polling	11
1.6.2 Using External Interrupts for Timing	11
1.6.3 Using Timer Interrupts	12
1.6.4 Ballast Coding	12
1.6.5 Using an External Real-Time Clock	13
1.7 Sensors Used in Computer Control	14
1.7.1 Temperature Sensors	15
1.7.2 Position Sensors	17
1.7.3 Velocity and Acceleration Sensors	20
1.7.4 Force Sensors	21
1.7.5 Pressure Sensors	21
1.7.6 Liquid Sensors	22
1.7.7 Air Flow Sensors	23
1.8 Exercises	24
Further Reading	25
2 System Modelling	27
2.1 Mechanical Systems	27
2.1.1 Translational Mechanical Systems	28
2.1.2 Rotational Mechanical Systems	32

2.2	Electrical Systems	37
2.3	Electromechanical Systems	42
2.4	Fluid Systems	44
2.4.1	Hydraulic Systems	44
2.5	Thermal Systems	49
2.6	Exercises	52
	Further Reading	52
3	The PIC Microcontroller	57
3.1	The PIC Microcontroller Family	57
3.1.1	The 10FXXX Family	58
3.1.2	The 12CXXX/PIC12FXXX Family	59
3.1.3	The 16C5X Family	59
3.1.4	The 16CXXX Family	59
3.1.5	The 17CXXX Family	60
3.1.6	The PIC18CXXX Family	60
3.2	Minimum PIC Configuration	61
3.2.1	External Oscillator	63
3.2.2	Crystal Operation	63
3.2.3	Resonator Operation	63
3.2.4	RC Operation	65
3.2.5	Internal Clock	65
3.3	Some Popular PIC Microcontrollers	66
3.3.1	PIC16F84 Microcontroller	67
3.3.2	PIC16F877 Microcontroller	71
3.4	Exercises	75
	Further Reading	76
4	Programming PIC Microcontrollers in C	77
4.1	PICC Lite Variable Types	78
4.1.1	Bit	78
4.1.2	Unsigned Char	78
4.1.3	Signed Char	79
4.1.4	Unsigned Int	79
4.1.5	Signed Int	79
4.1.6	Long	79
4.1.7	Unsigned Long	79
4.1.8	Float	80
4.1.9	Double	80
4.2	Variables	80
4.3	Comments in Programs	81
4.4	Storing Variables in the Program Memory	82
4.5	Static Variables	82
4.6	Volatile Variables	83
4.7	Persistent Variables	83

4.8	Absolute Address Variables	83
4.9	Bank1 Qualifier	83
4.10	Arrays	84
4.11	ASCII Constants	86
4.12	Arithmetic and Logic Operators	86
4.13	Number Bases	89
4.14	Structures	89
4.15	Program Flow Control	91
4.15.1	If–Else Statement	91
4.15.2	Switch–Case Statement	92
4.15.3	For Statement	94
4.15.4	While Statement	95
4.15.5	Do Statement	95
4.15.6	Break Statement	96
4.15.7	Continue Statement	96
4.16	Functions in C	96
4.16.1	User Functions	97
4.16.2	Built-in Functions	98
4.17	Pointers in C	99
4.18	Pre-processor Commands	101
4.18.1	#define	101
4.18.2	#include	103
4.18.3	#asm and #endasm	103
4.19	Accessing the EEPROM Memory	104
4.20	Interrupts in C Programs	104
4.21	Delays in C Programs	105
4.22	Structure of a C Program	105
4.23	PIC Microcontroller Input–Output Interface	107
4.23.1	Connecting an LED	107
4.23.2	Connecting a Push-Button Switch	109
4.23.3	Connecting an LCD	111
4.24	Exercises	116
	Further Reading	117
5	Microcontroller Project Development	119
5.1	Hardware and Software Requirements	119
5.2	Program Development Tools	120
5.2.1	Flow Charts	121
5.2.2	Structure Charts	121
5.2.3	Pseudocode	123
5.3	Exercise	129
	Further Reading	129
6	Sampled Data Systems and the z-Transform	131
6.1	The Sampling Process	131
6.2	The z-Transform	136

6.2.1	Unit Step Function	137
6.2.2	Unit Ramp Function	137
6.2.3	Exponential Function	138
6.2.4	General Exponential Function	138
6.2.5	Sine Function	139
6.2.6	Cosine Function	139
6.2.7	Discrete Impulse Function	140
6.2.8	Delayed Discrete Impulse Function	140
6.2.9	Tables of z-Transforms	140
6.2.10	The z-Transform of a Function Expressed as a Laplace Transform	140
6.2.11	Properties of z-Transforms	143
6.2.12	Inverse z-Transforms	145
6.3	Pulse Transfer Function and Manipulation of Block Diagrams	154
6.3.1	Open-Loop Systems	154
6.3.2	Open-Loop Time Response	156
6.3.3	Closed-Loop Systems	162
6.3.4	Closed-Loop Time Response	166
6.4	Exercises	166
	Further Reading	169
7	System Time Response Characteristics	171
7.1	Time Response Comparison	171
7.2	Time Domain Specifications	174
7.3	Mapping the s-Plane into the z-Plane	177
7.4	Damping Ratio and Undamped Natural Frequency in the z-Plane	178
7.4.1	Damping Ratio	178
7.4.2	Undamped Natural Frequency	179
7.5	Damping Ratio and Undamped Natural Frequency Using Formulae	181
7.6	Exercises	183
	Further Reading	184
8	System Stability	187
8.1	Factorizing the Characteristic Equation	187
8.2	Jury's Stability Test	189
8.3	Routh–Hurwitz Criterion	192
8.4	Root Locus	194
8.5	Nyquist Criterion	201
8.6	Bode Diagrams	205
8.7	Exercises	208
	Further Reading	211
9	Discrete Controller Design	213
9.1	Digital Controllers	214
9.1.1	Dead-Beat Controller	215
9.1.2	Dahlin Controller	217

9.1.3 Pole-Placement Control – Analytical	219
9.1.4 Pole-Placement Control – Graphical	222
9.2 PID Controller	230
9.2.1 Saturation and Integral Wind-Up	233
9.2.2 Derivative Kick	233
9.2.3 PID Tuning	134
9.3 Exercises	137
Further Reading	240
10 Controller Realization	243
10.1 Direct Structure	243
10.1.1 Direct Canonical Structure	243
10.1.2 Direct Noncanonical Structure	245
10.2 Cascade Realization	246
10.3 Parallel Realization	249
10.4 PID Controller Implementations	250
10.5 Microcontroller Implementations	253
10.5.1 Implementing Second-Order Modules	254
10.5.2 Implementing First-Order Modules	260
10.5.3 Implementing Higher-Order Modules	263
10.6 Choice of Sampling Interval	263
10.7 Exercises	267
Further Reading	268
11 Liquid Level Digital Control System: a Case Study	269
11.1 The System Schematic	269
11.2 System Model	270
11.3 Identification of the System	273
11.4 Designing a Controller	274
11.5 Conclusions	278
Appendix A Table of z-Transforms	283
Appendix B MATLAB Tutorial	285
Index	307

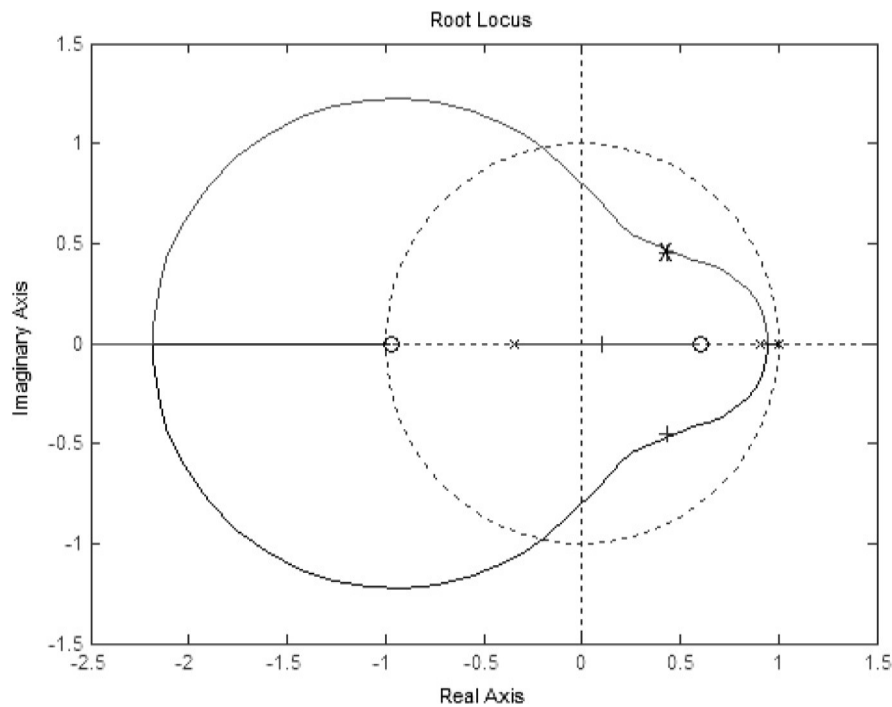


Figure 9.21 Root locus of the compensated system

Figure 9.21 shows the root locus of the compensated system. Clearly the locus passes through the required point. The d.c. gain at this point is $K = 123.9$.

The time response of the compensated system is shown in Figure 9.22.

9.2 PID CONTROLLER

The proportional–integral–derivative (PID) controller is often referred to as a ‘three-term’ controller. It is currently one of the most frequently used controllers in the process industry. In a PID controller the control variable is generated from a term proportional to the error, a term which is the integral of the error, and a term which is the derivative of the error.

- *Proportional*: the error is multiplied by a gain K_p . A very high gain may cause instability, and a very low gain may cause the system to drift away.
- *Integral*: the integral of the error is taken and multiplied by a gain K_i . The gain can be adjusted to drive the error to zero in the required time. A too high gain may cause oscillations and a too low gain may result in a sluggish response.
- *Derivative*: The derivative of the error is multiplied by a gain K_d . Again, if the gain is too high the system may oscillate and if the gain is too low the response may be sluggish.

Figure 9.23 shows the block diagram of the classical continuous-time PID controller. Tuning the controller involves adjusting the parameters K_p , K_d and K_i in order to obtain a satisfactory

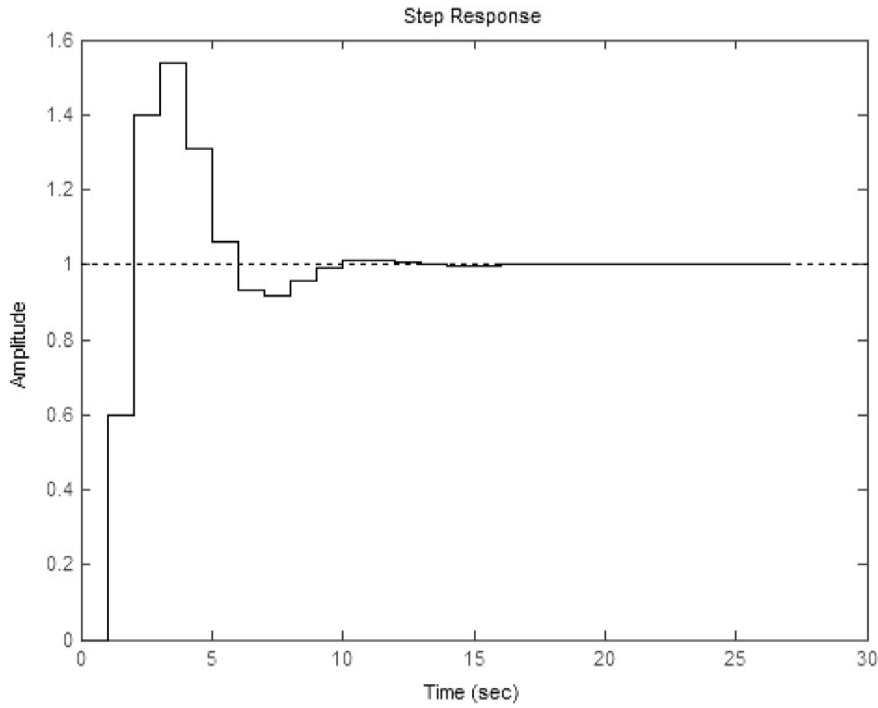


Figure 9.22 Time response of the system

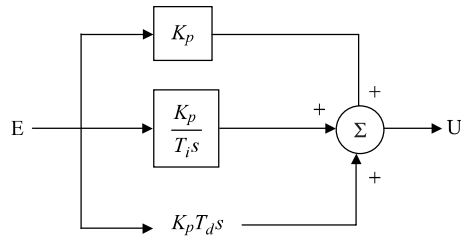


Figure 9.23 Continuous-time system PID controller

response. The characteristics of PID controllers are well known and well established, and most modern controllers are based on some form of PID.

The input–output relationship of a PID controller can be expressed as

$$u(t) = K_p \left[e(t) + \frac{1}{T_i} \int_0^t e(t) dt + T_d \frac{de(t)}{dt} \right], \quad (9.14)$$

where $u(t)$ is the output from the controller and $e(t) = r(t) - y(t)$, in which $r(t)$ is the desired set-point (reference input) and $y(t)$ is the plant output. T_i and T_d are known as the integral and

derivative action time, respectively. Notice that (9.14) is sometimes written as

$$u(t) = K_p e(t) + K_i \int_0^t e(t) dt + K_d \frac{de(t)}{dt} + u_0, \quad (9.15)$$

where

$$K_i = \frac{K_p}{T_i} \quad \text{and} \quad K_d = K_p T_d. \quad (9.16)$$

Taking the Laplace transform of (9.14), we can write the transfer function of a continuous-time PID as

$$\frac{U(s)}{E(s)} = K_p + \frac{K_p}{T_i s} + K_p T_d s. \quad (9.17)$$

To implement the PID controller using a digital computer we have to convert (9.14) from a continuous to a discrete representation. There are several methods for doing this and the simplest is to use the trapezoidal approximation for the integral and the backward difference approximation for the derivative:

$$\frac{de(t)}{dt} \approx \frac{e(kT) - e(kT - T)}{T} \quad \text{and} \quad \int_0^t e(t) dt \approx \sum_{k=1}^n T e(kT).$$

Equation (9.14) thus becomes

$$u(kT) = K_p \left[e(kT) + T_d \frac{e(kT) - e(kT - T)}{T} + \frac{T}{T_i} \sum_{k=1}^n e(kT) \right] + u_0. \quad (9.18)$$

The PID given by (9.18) is now in a suitable form which can be implemented on a digital computer. This form of the PID controller is also known as the *positional* PID controller. Notice that a new control action is implemented at every sample time.

The discrete form of the PID controller can also be derived by finding the z -transform of (9.17):

$$\frac{U(z)}{E(z)} = K_p \left[1 + \frac{T}{T_i(1 - z^{-1})} + T_d \frac{(1 - z^{-1})}{T} \right]. \quad (9.19)$$

Expanding (9.19) gives

$$\begin{aligned} u(kT) = u(kT - T) &+ K_p [e(kT) - e(kT - T)] + \frac{K_p T}{T_i} e(kT) \\ &+ \frac{K_p T_d}{T} [e(kT) - 2e(kT - T) + e(kT - 2T)]. \end{aligned} \quad (9.20)$$

This form of the PID controller is known as the *velocity* PID controller. Here the current control action uses the previous control value as a reference. Because only a change in the control action is used, this form of the PID controller provides a smoother bumpless control when the error is small. If a large error exists, the response of the velocity PID controller may be slow, especially if the integral action time T_i is large.

The two forms of the PID algorithm, (9.18) and (9.20), may look quite different, but they are in fact similar to each other. Consider the positional controller (9.18). Shifting back one

sampling interval, we obtain

$$u(kT - T) = K_p \left[e(kT - T) + T_d \frac{e(kT - T) - e(kT - 2T)}{T} + \frac{T}{T_i} \sum_{k=1}^{n-1} e(kT) \right] + u_0.$$

Subtracting from (9.18), we obtain the velocity form of the controller, as given by (9.20).

9.2.1 Saturation and Integral Wind-Up

In practical applications the output value of a control action is limited by physical constraints. For example, the maximum voltage output from a device is limited. Similarly, the maximum flow rate that a pump can supply is limited by the physical capacity of the pump. As a result of this physical limitation, the error signal does not return to zero and the integral term keeps adding up continuously. This effect is called integral wind-up (or integral saturation), and as a result of it long periods of overshoot can occur in the plant response. A simple example of what happens is the following. Suppose we wish to control the position of a motor and a large set-point change occurs, resulting in a large error signal. The controller will then try to reduce the error between the set-point and the output. The integral term will grow by summing the error signals at each sample and a large control action will be applied to the motor. But because of the physical limitation of the motor electronics the motor will not be able to respond linearly to the applied control signal. If the set-point now changes in the other direction, then the integral term is still large and will not respond immediately to the set-point request. Consequently, the system will have a poor response when it comes out of this condition.

The integral wind-up problem affects positional PID controllers. With velocity PID controllers, the error signals are not summed up and as a result integral wind-up will not occur, even though the control signal is physically constrained.

Many techniques have been developed to eliminate integral wind-up from the PID controllers, and some of the popular ones are as follows:

- Stop the integral summation when saturation occurs. This is also called conditional integration. The idea is to set the integrator input to zero if the controller output is saturated and the input and output are of the same sign.
- Fix the limits of the integral term between a minimum and a maximum.
- Reduce the integrator input by some constant if the controller output is saturated. Usually the integral value is decreased by an amount proportional to the difference between the unsaturated and saturated (i.e maximum) controller output.
- Use the velocity form of the PID controller.

9.2.2 Derivative Kick

Another possible problem when using PID controllers is caused by the derivative action of the controller. This may happen when the set-point changes sharply, causing the error signal to change suddenly. Under such a condition, the derivative term can give the output a *kick*, known as a *derivative kick*. This is usually avoided in practice by moving the derivative term

to the feedback loop. The proportional term may also cause a sudden kick in the output and it is common to move the proportional term to the feedback loop.

9.2.3 PID Tuning

When a PID controller is used in a system it is important to tune the controller to give the required response. Tuning a PID controller involves selecting values for the controller parameters K_p , T_i and T_d . There are many techniques for tuning a controller, ranging from the first techniques described by J.G. Ziegler and N.B. Nichols (known as the Ziegler–Nichols tuning algorithm) in 1942 and 1943, to recent auto-tuning controllers. In this section we shall look at the tuning of PID controllers using the Ziegler–Nichols tuning algorithm.

Ziegler and Nichols suggested values for the PID parameters of a plant based on open-loop or closed-loop tests of the plant. According to Ziegler and Nichols, the open-loop transfer function of a system can be approximated with a time delay and a single-order system, i.e.

$$G(s) = \frac{K e^{-sT_D}}{1 + sT_1}, \quad (9.21)$$

where T_D is the system time delay (i.e. transportation delay), and T_1 is the time constant of the system.

9.2.3.1 Open-Loop Tuning

For open-loop tuning, we first find the plant parameters by applying a step input to the open-loop system. The plant parameters K , T_D and T_1 are then found from the result of the step test as shown in Figure 9.24.

Ziegler and Nichols then suggest using the PID controller settings given in Table 9.1 when the loop is closed. These parameters are based on the concept of minimizing the integral of the absolute error after applying a step change to the set-point.

An example is given below to illustrate the method used.

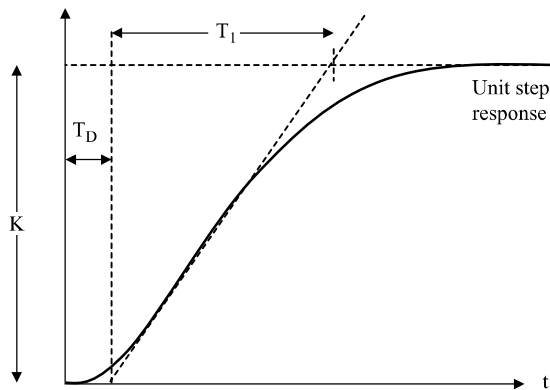


Figure 9.24 Finding plant parameters K , T_D and T_1

Table 9.1 Open-loop Ziegler–Nichols settings

Controller	K_p	T_i	T_d
Proportional	$\frac{T_1}{KT_D}$		
PI	$\frac{0.9T_1}{KT_D}$	$3.3T_D$	
PID	$\frac{1.2T_1}{KT_D}$	$2T_D$	$0.5T_D$

Example 9.7

The open-loop unit step response of a thermal system is shown in Figure 9.25. Obtain the transfer function of this system and use the Ziegler–Nichols tuning algorithm to design (a) a proportional controller, (b) to design a proportional plus integral (PI) controller, and (c) to design a PID controller. Draw the block diagram of the system in each case.

Solution

From Figure 9.25, the system parameters are obtained as $K = 40^\circ\text{C}$, $T_D = 5\text{ s}$ and $T_1 = 20\text{ s}$, and the transfer function of the plant is

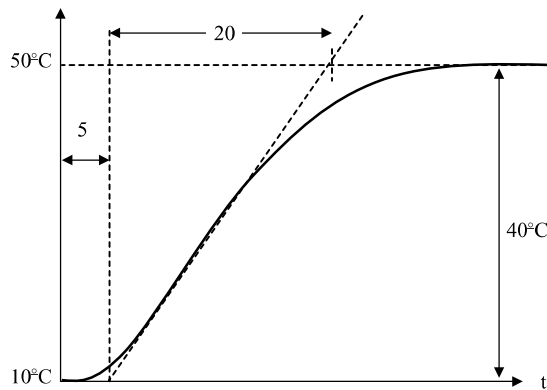
$$G(s) = \frac{40e^{-5s}}{1 + 20s}.$$

Proportional controller. According to Table 9.1, the Ziegler–Nichols settings for a proportional controller are:

$$K_p = \frac{T_1}{KT_D}.$$

Thus,

$$K_p = \frac{20}{40 \times 5} = 0.1,$$

**Figure 9.25** Unit step response of the system for Example 9.7